

Step 1: Create the Customer table with customer ID, name, phone number and email details.

The screenshot displays a database management interface. The top section shows a SQL script with the following content:

```
4 CREATE TABLE Customer (  
5     Customer_ID INT PRIMARY KEY,  
6     Customer_Name VARCHAR(100),  
7     Phone VARCHAR(15),  
8     Email VARCHAR(100)  
9 );  
10 INSERT INTO Customer  
11 (Customer_ID, Customer_Name, Phone, Email)  
12 VALUES  
13 (101, 'Rahul', '9876543210', 'rahul@gmail.com'),  
14 (102, 'Priya', '9876543211', 'priya@gmail.com'),  
15 (103, 'Arun', '9876543212', 'arun@gmail.com');
```

Below the script, the 'Result Grid' shows the data inserted into the 'Customer' table:

Customer_ID	Customer_Name	Phone	Email
101	Rahul	9876543210	rahul@gmail.com
102	Priya	9876543211	priya@gmail.com
103	Arun	9876543212	arun@gmail.com
104	Sneha	9876543213	sneha@gmail.com
105	Kiran	9876543214	kiran@gmail.com
NULL	NULL	NULL	NULL

On the right side, a message states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

The bottom section shows the 'Output' window with the following log:

#	Time	Action	Message	Duration / Fetch
1	18:15:07	CREATE TABLE Account (Account_No INT PRIMARY...	Error Code: 1824. Failed to open the referenced table 'cust...	0.016 sec
2	18:16:32	INSERT INTO Account (Account_No, Customer_ID, Acco...	Error Code: 1146. Table 'p_week1.account' doesn't exist	0.000 sec
3	18:17:18	CREATE TABLE Customer (Customer_ID INT PRIMAR...	0 row(s) affected	0.031 sec
4	18:17:31	INSERT INTO Customer (Customer_ID, Customer_Name, ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
5	18:17:45	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Step 2: Verify the Customer table and display the inserted customer records.

```
19
20 • CREATE TABLE Account (
21     Account_No INT PRIMARY KEY,
22     Customer_ID INT,
23     Account_Type VARCHAR(20),
24     Balance DECIMAL(12,2),
25     Branch VARCHAR(50),
26     FOREIGN KEY (Customer_ID)
27         REFERENCES Customer(Customer_ID)
28 );
29 • SELECT * FROM Account;
```

Result Grid



Filter Rows:

Edit:



Export/Import:



	Account_No	Customer_ID	Account_Type	Balance	Branch
*	NULL	NULL	NULL	NULL	NULL

Account 2

Apply

Output



Action Output

	#	Time	Action	Message
✗	2	18:16:32	INSERT INTO Account (Account_No, Customer_ID, Acc...	Error Code: 1146. Tab
✓	3	18:17:18	CREATE TABLE Customer (Customer_ID INT PRIMA...	0 row(s) affected
✓	4	18:17:31	INSERT INTO Customer (Customer_ID, Customer_Name,...	5 row(s) affected Recd
✓	5	18:17:45	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
✓	6	18:18:27	CREATE TABLE Account (Account_No INT PRIMAR...	0 row(s) affected
✓	7	18:18:47	SELECT * FROM Account LIMIT 0, 1000	0 row(s) returned

Step 3: Create the Account table and connect it to Customer using a foreign key.

```
40 • CREATE TABLE Loan (  
41     Loan_ID INT PRIMARY KEY,  
42     Customer_ID INT,  
43     Loan_Type VARCHAR(30),  
44     Loan_Amount DECIMAL(12,2),  
45     Interest_Rate DECIMAL(5,2),  
46     FOREIGN KEY (Customer_ID)  
47     REFERENCES Customer(Customer_ID)  
48 );  
49 • SHOW TABLES;  
50
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Tables_in_p_week1
▶	account
	bank_transaction
	books
	customer
	loan
	members
	student_marks1

Result 3 x

Output

Action Output

	#	Time	Action	Message
✓	5	18:17:45	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
✓	6	18:18:27	CREATE TABLE Account (Account_No INT PRIMAR...	0 row(s) affected
✓	7	18:18:47	SELECT * FROM Account LIMIT 0, 1000	0 row(s) returned
✓	8	18:19:55	CREATE TABLE Bank_Transaction (Transaction_ID I...	0 row(s) affected
✓	9	18:20:10	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY,...	0 row(s) affected
✓	10	18:20:10	SHOW TABLES	7 row(s) returned

Step 4: Create the Bank_Transaction and Loan tables for banking transactions and loan details.

```
50
51 • INSERT INTO Account
52 (Account_No, Customer_ID, Account_Type, Balance, Branch)
53 VALUES
54 (10001, 101, 'Savings', 50000, 'Hyderabad'),
55 (10002, 102, 'Savings', 75000, 'Vijayawada'),
56 (10003, 103, 'Current', 120000, 'Bangalore'),
57 (10004, 104, 'Savings', 45000, 'Chennai'),
58 (10005, 105, 'Current', 98000, 'Hyderabad');
59 • INSERT INTO Bank_Transaction
60 (Account_No, Transaction_Type, Amount)
61 VALUES
62 (10001, 'DEPOSIT', 10000),
63 (10002, 'DEPOSIT', 15000),
64 (10003, 'WITHDRAW', 20000),
65 (10004, 'DEPOSIT', 5000),
66 (10005, 'WITHDRAW', 10000);
```

Output



Action Output

	#	Time	Action	Message
✓	7	18:18:47	SELECT * FROM Account LIMIT 0, 1000	0 row(s) returned
✓	8	18:19:55	CREATE TABLE Bank_Transaction (Transaction_ID I...	0 row(s) affected
✓	9	18:20:10	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY,...	0 row(s) affected
✓	10	18:20:10	SHOW TABLES	7 row(s) returned
✓	11	18:20:53	INSERT INTO Account (Account_No, Customer_ID, Acc...	5 row(s) affected Records: 5 Dupli
✓	12	18:21:12	INSERT INTO Bank_Transaction (Account_No, Transac...	5 row(s) affected Records: 5 Dupli

Step 5: Insert account records with account type, balance and branch information.

```
71      (502, 102, 'Education Loan', 1000000, 6.5),
72      (503, 103, 'Car Loan', 800000, 8.2),
73      (504, 104, 'Personal Loan', 500000, 10.5);
74 •    SELECT * FROM Customer;
75
76 •    SELECT * FROM Account;
77
78 •    SELECT * FROM Bank_Transaction;
79
80 •    SELECT * FROM Loan;
```

Result Grid | | Filter Rows: | Edit: | Export/Import:

	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	501	101	Home Loan	5000000.00	7.50
	502	102	Education Loan	1000000.00	6.50
	503	103	Car Loan	800000.00	8.20
	504	104	Personal Loan	500000.00	10.50
*	NULL	NULL	NULL	NULL	NULL

Customer 4 Account 5 Bank_Transaction 6 **Loan 7** ×

Output

Action Output ▼

	#	Time	Action	Message
✓	12	18:21:12	INSERT INTO Bank_Transaction (Account_No, Transac...	5 row(s) affected Recd
✓	13	18:21:40	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type...	4 row(s) affected Recd
✓	14	18:21:57	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
✓	15	18:21:57	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
✓	16	18:21:57	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
✓	17	18:21:57	SELECT * FROM Loan LIMIT 0, 1000	4 row(s) returned

Step 6: Insert deposit and withdrawal transaction records for the accounts.

```
79 DELIMITER //
80 • CREATE PROCEDURE GetAllCustomers()
81 • BEGIN
82 •     SELECT * FROM Customer;
83 • END //
84 DELIMITER ;
85 • CALL GetAllCustomers();
```

ma
cur
to

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Customer_ID	Customer_Name	Phone	Email
▶	101	Rahul	9876543210	rahul@gmail.com
	102	Priya	9876543211	priya@gmail.com
	103	Arun	9876543212	arun@gmail.com
	104	Sneha	9876543213	sneha@gmail.com
	105	Kiran	9876543214	kiran@gmail.com

Result Grid
Form Editor

Result 8 x Read Only Context

Output

Action Output

#	Time	Action	Message
✓ 15	18:21:57	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
✓ 16	18:21:57	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
✓ 17	18:21:57	SELECT * FROM Loan LIMIT 0, 1000	4 row(s) returned
✗ 18	18:22:45	CREATE PROCEDURE GetAllCustomers() BEGIN SEL...	Error Code: 1064. You have an error in your S
✓ 19	18:22:56	CREATE PROCEDURE GetAllCustomers() BEGIN SEL...	0 row(s) affected
✓ 20	18:22:56	CALL GetAllCustomers()	5 row(s) returned

Step 7: Insert loan records containing loan type, amount and interest rate.

```
88 • CREATE PROCEDURE GetAccountDetails(  
89     IN p_Account_No INT  
90 )  
91 BEGIN  
92     SELECT *  
93     FROM Account  
94     WHERE Account_No = p_Account_No;  
95 END //  
96 DELIMITER ;  
97 • CALL GetAccountDetails(10001);
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

Form Editor

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad

Result 9 × Read Only

Output

Action Output

	#	Time	Action	Message
✓	17	18:21:57	SELECT * FROM Loan LIMIT 0, 1000	4 row(s) returned
✗	18	18:22:45	CREATE PROCEDURE GetAllCustomers() BEGIN SEL...	Error Code: 1064. You have an en
✓	19	18:22:56	CREATE PROCEDURE GetAllCustomers() BEGIN SEL...	0 row(s) affected
✓	20	18:22:56	CALL GetAllCustomers()	5 row(s) returned
✓	21	18:23:45	CREATE PROCEDURE GetAccountDetails(IN p_Acc...	0 row(s) affected
✓	22	18:23:45	CALL GetAccountDetails(10001)	1 row(s) returned

Step 8: Display the Customer, Account, Bank_Transaction and Loan records using SELECT queries.

```
108         A.Account_Type,  
109         A.Balance,  
110         A.Branch  
111     FROM Customer C  
112     JOIN Account A  
113         ON C.Customer_ID = A.Customer_ID  
114     WHERE C.Customer_ID = p_Customer_ID;  
115 END //  
116 DELIMITER ;  
117 • CALL GetCustomerAccounts(101);
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Result
Grid



Form
Editor



	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Rahul	10001	Savings	50000.00	Hyderabad

Result 10 x

Read Only

Output



Action Output



	#	Time	Action	Message
✓	19	18:22:56	CREATE PROCEDURE GetAllCustomers() BEGIN SEL...	0 row(s) affected
✓	20	18:22:56	CALL GetAllCustomers()	5 row(s) returned
✓	21	18:23:45	CREATE PROCEDURE GetAccountDetails(IN p_Acc...	0 row(s) affected
✓	22	18:23:45	CALL GetAccountDetails(10001)	1 row(s) returned
✓	23	18:24:28	CREATE PROCEDURE GetCustomerAccounts(IN p_...	0 row(s) affected
✓	24	18:24:28	CALL GetCustomerAccounts(101)	1 row(s) returned

Step 9: Create and execute GetAllCustomers() to retrieve all customer details.

```
124 BEGIN
125     UPDATE Account
126     SET Balance = Balance + p_Amount
127     WHERE Account_No = p_Account_No;
128 END //
129 DELIMITER ;
130 • CALL DepositMoney(10001, 5000);
131 • SELECT *
132 FROM Account
133 WHERE Account_No = 10001;
```

Result Grid



Filter Rows:

Edit:



Export/Import:



Result Grid

Form Editor



	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	55000.00	Hyderabad
•	NULL	NULL	NULL	NULL	NULL

Account 11 x

Apply

Output



Action Output

	#	Time	Action	Message
✓	22	18:23:45	CALL GetAccountDetails(10001)	1 row(s) returned
✓	23	18:24:28	CREATE PROCEDURE GetCustomerAccounts(IN p_...	0 row(s) affected
✓	24	18:24:28	CALL GetCustomerAccounts(101)	1 row(s) returned
✓	25	18:25:08	CREATE PROCEDURE DepositMoney(IN p_Account...	0 row(s) affected
✓	26	18:25:08	CALL DepositMoney(10001, 5000)	1 row(s) affected
✓	27	18:25:08	SELECT * FROM Account WHERE Account_No = 1000...	1 row(s) returned

Step 10: Create and execute GetAccountDetails() to retrieve details for a specified account.

```
151     NOW TRIGGERS
152     DELIMITER //
153 •   CREATE TRIGGER CheckBalance
154     BEFORE UPDATE ON Account
155     FOR EACH ROW
156     BEGIN
157     IF NEW.Balance < 0 THEN
158         SIGNAL SQLSTATE '45000'
159         SET MESSAGE_TEXT =
160             'Transaction failed: Insufficient balance';
161     END IF;
162     END //
163     DELIMITER ;
164
```

[Context Help](#)

Output



Action Output

	#	Time	Action	Message
✓	26	18:25:08	CALL DepositMoney(10001, 5000)	1 row(s) affected
✓	27	18:25:08	SELECT * FROM Account WHERE Account_No = 1000...	1 row(s) returned
✓	28	18:25:52	CREATE PROCEDURE WithdrawMoney(IN p_Accou...	0 row(s) affected
✓	29	18:25:52	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
✓	30	18:25:52	SELECT * FROM Account WHERE Account_No = 1000...	1 row(s) returned
✓	31	18:26:55	CREATE TRIGGER CheckBalance BEFORE UPDATE ...	0 row(s) affected

Step 11: Create GetCustomerAccounts() to display the account details of a selected customer.

```
277         L.Loan_Type,  
278         L.Loan_Amount,  
279         L.Interest_Rate  
280     FROM Customer C  
281     JOIN Loan L  
282         ON C.Customer_ID = L.Customer_ID  
283     WHERE C.Customer_ID = p_Customer_ID;  
284 END //  
285 DELIMITER ;  
286 • CALL GetCustomerLoans(101);
```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
	Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	Rahul	501	Home Loan	5000000.00	7.50

Result Grid

Form Editor

Result 13 x

Read Only

Output



Action Output

	#	Time	Action	Message
✖	34	18:28:18	INSERT INTO Bank_Transaction (Account_No, Transac...	Error Code: 1146. Table 'p_week1.'
✔	35	18:28:51	CREATE TRIGGER UpdateAccountBalance AFTER INS...	0 row(s) affected
✔	36	18:29:28	CREATE TRIGGER PreventInsufficientWithdrawal BEFO...	0 row(s) affected
✔	37	18:30:10	CREATE PROCEDURE TransferMoney(IN SenderAc...	0 row(s) affected
✔	38	18:30:40	CREATE PROCEDURE GetCustomerLoans(IN p_Cus...	0 row(s) affected
✔	39	18:30:40	CALL GetCustomerLoans(101)	1 row(s) returned

Step 12: Create DepositMoney() and update the account balance after depositing money.

```
290      IN MinimumBalance DECIMAL(12,2)
291    )
292  BEGIN
293      SELECT *
294      FROM Account
295      WHERE Balance >= MinimumBalance
296      ORDER BY Balance DESC;
297  END //
298  DELIMITER ;
299  • CALL HighBalanceAccounts(50000);
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Result Grid
Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	98000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10001	101	Savings	52000.00	Hyderabad

Result 14 x

Read Only

Output

Action Output

#	Time	Action	Message
✓ 36	18:29:28	CREATE TRIGGER PreventInsufficientWithdrawal BEFO...	0 row(s) affected
✓ 37	18:30:10	CREATE PROCEDURE TransferMoney(IN SenderAc...	0 row(s) affected
✓ 38	18:30:40	CREATE PROCEDURE GetCustomerLoans(IN p_Cus...	0 row(s) affected
✓ 39	18:30:40	CALL GetCustomerLoans(101)	1 row(s) returned
✓ 40	18:31:21	CREATE PROCEDURE HighBalanceAccounts(IN Mi...	0 row(s) affected
✓ 41	18:31:21	CALL HighBalanceAccounts(50000)	4 row(s) returned

Step 13: Create balance-checking and transaction triggers to validate banking operations.

305)

306 BEGIN

307 SELECT Balance

308 INTO p_Balance

309 FROM Account

310 WHERE Account_No = p_Account_No;

311 END //

312 DELIMITER ;

313 • CALL GetBalance(10001, @CurrentBalance);

314 • SELECT @CurrentBalance;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

Form Editor

Read Only

Result 15

Output

Action Output

#	Time	Action	Message
✓ 39	18:30:40	CALL GetCustomerLoans(101)	1 row(s) returned
✓ 40	18:31:21	CREATE PROCEDURE HighBalanceAccounts(IN Mi...	0 row(s) affected
✓ 41	18:31:21	CALL HighBalanceAccounts(50000)	4 row(s) returned
✓ 42	18:32:11	CREATE PROCEDURE GetBalance(IN p_Account_N...	0 row(s) affected
✓ 43	18:32:11	CALL GetBalance(10001, @CurrentBalance)	1 row(s) affected
✓ 44	18:32:11	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned

Step 14: Create GetCustomerLoans() to retrieve loans belonging to a selected customer.

315
316 • `show triggers;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Trigger	Event	Table	Statement
▶	CheckBalance	UPDATE	account	BEGIN IF NEW.Balance < 0 TH
	CheckTransactionAmount	INSERT	bank_transaction	BEGIN IF NEW.Amount <= 0 T
	PreventInsufficientWithdrawal	INSERT	bank_transaction	BEGIN DECLARE CurrentBalanc
	TransactionAudit	INSERT	bank_transaction	BEGIN INSERT INTO Transacti
	UpdateAccountBalance	INSERT	bank_transaction	BEGIN IF NEW.Transaction_Ty

Result Grid
Form Editor

Result 16 x Read Only

Output

Action Output

	#	Time	Action	Message
✓	40	18:31:21	CREATE PROCEDURE HighBalanceAccounts(IN Mi...	0 row(s) affected
✓	41	18:31:21	CALL HighBalanceAccounts(50000)	4 row(s) returned
✓	42	18:32:11	CREATE PROCEDURE GetBalance(IN p_Account_N...	0 row(s) affected
✓	43	18:32:11	CALL GetBalance(10001, @CurrentBalance)	1 row(s) affected
✓	44	18:32:11	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
✓	45	18:32:41	show triggers	5 row(s) returned

Step 15: Create HighBalanceAccounts() and GetBalance() to retrieve and check account balances.

```
319 • CREATE PROCEDURE GetBranchAccounts(  
320     IN p_Branch VARCHAR(50)  
321 )  
322 BEGIN  
323     SELECT *  
324     FROM Account  
325     WHERE Branch = p_Branch;  
326 END //  
327 DELIMITER ;  
328 • CALL GetBranchAccounts('Hyderabad');
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

Result Grid

Form Editor

Read Only

Result 17 x

Output



Action Output

#	Time	Action	Message
✓ 42	18:32:11	CREATE PROCEDURE GetBalance(IN p_Account_N...	0 row(s) affected
✓ 43	18:32:11	CALL GetBalance(10001, @CurrentBalance)	1 row(s) affected
✓ 44	18:32:11	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
✓ 45	18:32:41	show triggers	5 row(s) returned
✓ 46	18:33:38	CREATE PROCEDURE GetBranchAccounts(IN p_Br...	0 row(s) affected
✓ 47	18:33:38	CALL GetBranchAccounts('Hyderabad')	2 row(s) returned

Step 16: Display the available triggers and create GetBranchAccounts() for branch-wise accounts.

```
339         A.Account_No,
340         A.Account_Type,
341         A.Balance
342     FROM Customer C
343     JOIN Account A
344         ON C.Customer_ID = A.Customer_ID
345     WHERE C.Customer_ID = p_Customer_ID;
346 END //
347 DELIMITER ;
348 • CALL GetCustomerFullDetails(101);
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

Form Editor

	Customer_Name	Phone	Email	Account_No	Account_Type	Balance
▶	Rahul	9876543210	rahul@gmail.com	10001	Savings	52000.00

Result 18 x Read Only

Output

Action Output

	#	Time	Action	Message
✓	44	18:32:11	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
✓	45	18:32:41	show triggers	5 row(s) returned
✓	46	18:33:38	CREATE PROCEDURE GetBranchAccounts(IN p_Br...	0 row(s) affected
✓	47	18:33:38	CALL GetBranchAccounts('Hyderabad')	2 row(s) returned
✓	48	18:34:39	CREATE PROCEDURE GetCustomerFullDetails(IN p_...	0 row(s) affected
✓	49	18:34:39	CALL GetCustomerFullDetails(101)	1 row(s) returned

Step 17: Create GetCustomerFullDetails() and GetTotalCustomerBalance() to retrieve complete customer and balance information.

354 BEGIN

355 SELECT

356 Customer_ID,

357 SUM(Balance) AS Total_Balance

358 FROM Account

359 WHERE Customer_ID = p_Customer_ID

360 GROUP BY Customer_ID;

361 END //

362 DELIMITER ;

363 CALL GetTotalCustomerBalance(101);

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

Form Editor

	Customer_ID	Total_Balance
▶	101	52000.00

Result 19 × Read Only

Output

Action Output

	#	Time	Action	Message
✓	46	18:33:38	CREATE PROCEDURE GetBranchAccounts(IN p_Br...	0 row(s) affected
✓	47	18:33:38	CALL GetBranchAccounts('Hyderabad')	2 row(s) returned
✓	48	18:34:39	CREATE PROCEDURE GetCustomerFullDetails(IN p_...	0 row(s) affected
✓	49	18:34:39	CALL GetCustomerFullDetails(101)	1 row(s) returned
✓	50	18:40:53	CREATE PROCEDURE GetTotalCustomerBalance(IN...	0 row(s) affected
✓	51	18:40:53	CALL GetTotalCustomerBalance(101)	1 row(s) returned